Nome e Cognome: _____ **Matricola:** _____

Istruzioni: Il tempo a disposizione per completare la prova è di **45 minuti**. Durante la prova, non è consentito consultare materiale didattico, né interagire con dispositivi elettronici o colleghi. Nel caso di domande a risposta multipla con più risposte corrette, ciascuna risposta corretta indicata vale una corrispondente frazione del punteggio complessivo, mentre le risposte errate comportano una pari penalizzazione. In nessun caso, il punteggio complessivo relativo a una domanda sarà negativo.

Es. **1.** **(2 punti)** Nel protocollo HTTP, come si svolge la comunicazione fra client e server?

- A Il server apre la comunicazione inviando l'elenco delle risorse disponibili.
- B Il client si collega e attende che il server scelga quale pagina inviare.
- C Il server risponde solo ai client che hanno già una sessione aperta.
- D Il server invia periodicamente le pagine aggiornate ai client collegati.
- **E** Il client invia una richiesta e il server risponde; il server non inizia mai.
- F Client e server possono iniziare la comunicazione indifferentemente.

Risposta: E. HTTP è un protocollo di tipo richiesta-risposta: il client parla per primo, sempre. Il server si limita a rispondere alle richieste che riceve. *(Dispensa: cap. 1, «HTTP: il protocollo del Web»)*

Es. **2.** **(2 punti)** Quale dei seguenti frammenti HTML è annidato **correttamente**?

- A `<p>Un <em>testo</p> molto importante</em>.`
- B `<em><p>Un testo</em> molto importante.</p>`
- C `<p>Un <em>testo <strong>molto</em> importante</strong>.</p>`
- **D** `<p>Un <em>testo <strong>molto</strong> importante</em>.</p>`

Risposta: D. Il documento è un albero: un elemento aperto dentro un altro deve chiudersi prima del suo contenitore. Tag «incrociati» non sono validi e il browser li corregge a modo suo. *(Dispensa: cap. 2, «Il documento come albero»)*

Es. **3.** **(2 punti)** Che effetto ha l'assenza di `<meta name="viewport" content="width=device-width, initial-scale=1">` in una pagina responsive vista su uno smartphone?

- **A** Il browser impagina la pagina su un viewport largo come un desktop e la rimpicciolisce.
- B Le immagini vengono scaricate in bassa risoluzione.
- C Nessuno: il tag serve solo ai motori di ricerca.
- D Le media query vengono ignorate solo sui browser desktop.
- E La pagina non viene visualizzata e il browser mostra un errore.
- F Il browser carica automaticamente la versione mobile del sito.

Risposta: A. Senza il meta tag i browser mobili simulano uno schermo largo circa 980px e poi riducono tutto: le media query pensate per gli schermi stretti non scattano e il testo risulta minuscolo. `width=device-width` fa coincidere il viewport con la larghezza reale del dispositivo. *(Dispensa: cap. 4, «Il meta tag viewport»)*

Es. **4.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1 console.log(typeof Pet);
2 const p = new Pet("Fido");
3 class Pet {
4   constructor(nome) { this.nome = nome; }
5 }
```

- **A** Viene sollevato un `ReferenceError`.
- B `undefined`, poi viene sollevato un `TypeError`.
- C `function`.
- D `class`.
- E `undefined`.
- F `function`, poi viene sollevato un `ReferenceError`.

Risposta: A. Le dichiarazioni di classe vengono sollevate come `let`: fino alla dichiarazione la classe è nella *temporal dead zone*. Già `typeof Pet` solleva un `ReferenceError` (lo stesso `typeof` che, per un nome mai dichiarato, restituirebbe `"undefined"`). *(Dispensa: cap. 6, «Non solo zucchero sintattico»)*

SOLUZIONI

Es. **5.** **(2 punti)** Qual è l'output in console dopo il caricamento completo del documento HTML riportato di seguito?

```
1  <!DOCTYPE html>
2  <html lang="en">
3    <head>
4      <title>Web</title>
5      <meta charset="UTF-8"/></head>
6  <body>
7   <script>
8     let h = document.head?.children;
9     console.log(h[1]?.outerHTML);
10  </script></body></html>
```

- **A** `<meta charset="UTF-8">`.
- B Una stringa contenente solo spazi bianchi.
- C Viene sollevato un `TypeError`.
- D `null`.
- E `<title>Web</title>`.
- F `undefined`.

Risposta: A. `document.head.children` contiene solo gli **elementi** figli di `<head>`, senza i nodi di testo: `[0]` è `<title>`, `[1]` è `<meta>`. `outerHTML` è il codice dell'elemento ricostruito a partire dal DOM: la barra di `/>`, che per il parser non conta, non compare. *(Dispensa: cap. 7, «Navigare fra soli elementi») — Fonte: prima prova intercorso del 23 aprile 2024.*

Es. **6.** **(2 punti)** Quali delle seguenti affermazioni sull'oggetto `Response` restituito da `fetch` sono **vere**?

- A Il corpo si legge come stringa dalla proprietà `response.text`.
- B `response.ok` vale `true` per qualunque risposta del server.
- C `response.json()` restituisce direttamente l'oggetto, senza Promise.
- D `fetch` restituisce direttamente l'oggetto `Response`, senza Promise.
- **E** `response.json()` restituisce una Promise, che si risolve con il corpo interpretato come JSON.
- **F** La Promise di `fetch` si risolve appena arrivano gli header: il corpo può non essere ancora disponibile.

Risposte: E, F. `fetch` restituisce una Promise che si risolve con la `Response` quando arrivano gli header (`status`, `ok`, `headers`). Il corpo si legge dopo, con metodi che restituiscono a loro volta Promise (`text()`, `json()`, `blob()`...), e una volta sola, perché leggerlo lo consuma. *(Dispensa: cap. 8, «Lavorare con la risposta»)*

Es. **7.** **(2 punti)** Un contenitore flex ha `gap: 30px` e `padding: 10px`. Dove si trova lo spazio di 30px?

- **A** Solo fra un figlio e l'altro (anche fra le righe, se vanno a capo), mai fra i figli e il bordo del contenitore.
- B Fra il bordo e il contenuto del contenitore, come il padding.
- C Solo fra il primo figlio e il bordo sinistro del contenitore.
- D Solo fra le righe, quando i figli vanno a capo.
- E `gap` funziona solo con Grid: nel contenitore flex non ha effetto.
- F Attorno a ogni figlio, anche verso il bordo del contenitore.

Risposta: A. `gap` si usa sia con Grid sia con Flexbox e aggiunge spazio solo fra gli elementi; verso il bordo provvede il `padding` del contenitore. *(Dispensa: cap. 14, «gap con Grid e Flexbox»)*

SOLUZIONI

Es. **8.** **(2 punti)** Si consideri la seguente applicazione Express, in ascolto sulla porta 3000. Quali risposte (codice di stato e corpo) ottengono, nell'ordine, le richieste `GET /ok` e `GET /errore`?

```
1  import express from "express";
2
3  const app = express();
4
5  app.get("/ok", (req, res) => res.send("ok"));
6  app.get("/errore", (req, res) => {
7    throw new Error("Rotto");
8  });
9  app.use((err, req, res, next) => {
10   res.status(500).send("Gestito: " + err.message);
11 });
12
13 app.listen(3000);
```

A `200 ok, 500 Internal Server Error.`
B `200 ok, 200 Gestito: Rotto.`
C `200 ok, 500 Gestito: Rotto.`
D `500 Error: Rotto, 500 Error: Rotto.`

Risposta: C. L'errore sollevato nel codice sincrono di un gestore arriva al gestore di errori personalizzato, che ha quattro parametri ed è dichiarato dopo le rotte. Un gestore di errori si riconosce dall'**arità**: quattro parametri, `(err, req, res, next)`. *(Dispensa: cap. 15, «Gestione degli errori»)*

Es. **9.** **(5 punti)** Si spieghi come si calcola e come si confronta la **specificità** di un selettore CSS, con almeno tre esempi, e in quale punto dell'algoritmo della cascata interviene.

Traccia della risposta.

- Terna (a, b, c): a = id, b = classi, attributi e pseudo-classi, c = tipi di elemento e pseudo-elementi; `*` e combinatori non contano.
- Si confronta componente per componente da sinistra: non è un numero a tre cifre.
- Esempi: `h1` (0, 0, 1), `.nota p` (0, 1, 1), `#menu a:hover` (1, 1, 1).
- Interviene dopo origine/importanza e livelli (lo stile inline vince sui fogli di stile); a parità di specificità vince la dichiarazione che compare per ultima.

(Dispensa: cap. 3, «Specificità»)

Es. **10.** **(5 punti)** Nella lista di cose da fare realizzata con Node.js, che cosa succede se, dopo aver aggiunto un elemento con il form, l'utente ricarica la pagina? Si descriva il pattern *Post/Redirect/Get* e si spieghi perché cancellare un elemento con un semplice collegamento (`/delete?id=3`) non è una buona idea.

Traccia della risposta.

- La pagina mostrata è la risposta alla `POST`: ricaricandola, il browser chiede se reinviare il form, e l'elemento verrebbe salvato due volte.
- Post/Redirect/Get: il gestore della `POST` salva e risponde con un reindirizzamento (`303` e `Location: /todo`); il browser chiede da solo la lista con una `GET`, che si può ricaricare senza effetti.
- Una `GET` è un metodo sicuro: non deve modificare lo stato del server. Crawler, precaricamento dei link o estensioni potrebbero «visitare» `/delete?id=3` e cancellare l'elemento.
- La versione corretta usa un form con `method="POST"` e l'identificativo in un campo nascosto; il router deve guardare il `pathname` (`new URL(...)`), perché `request.url` contiene anche la query.

(Dispensa: cap. 12, «Mettere insieme i pezzi» ed «Estendere l'applicazione: cancellare un elemento»)

Es. **11.** **(5 punti)** Si descriva come realizzare l'autenticazione in un'applicazione Express con le sessioni e un middleware, e si spieghi il problema che nasce applicando il middleware con `router.use` a un router montato senza percorso.

Traccia della risposta.

- Al login (`POST`) il controller verifica le credenziali (password come hash) e scrive nella sessione `isAuthenticated` e il nome utente; il logout distrugge la sessione.
- Un middleware controlla la sessione e reindirizza al login (con un messaggio flash) chi non è autenticato, altrimenti chiama `next()`: un solo punto invece di un controllo in ogni controller.
- Si applica a tutte le rotte di un router (`router.use(mw)`) o a una sola rotta (`router.post("/reset", mw, gestore)`).
- Trappola: con `app.use(router)` e `router.use(mw)` senza percorso il middleware intercetta tutte le richieste non gestite prima (anche quelle per percorsi inesistenti, che non arrivano mai al 404); rimedio: `router.use("/todo", mw)` o `app.use("/todo", router)`.

(Dispensa: cap. 16, «Autenticazione» e «Una trappola: il middleware di un router»)

Griglia delle risposte

Es.	Risposta	Capitolo	Argomento
1	E	1	HTTP: richiesta e risposta
2	D	2	il documento come albero
3	A	4	meta viewport
4	A	6	classi
5	A	7	navigare il DOM
6	E, F	8	fetch
7	A	14	flex e gap
8	C	15	gestione degli errori
9	aperta	3	cascata e specificità
10	aperta	12	corpo delle richieste
11	aperta	16	autenticazione

Punteggio massimo: 16 (chiuse) + 15 (aperte) = **31 punti**. Nelle domande con più risposte corrette ogni risposta giusta indicata vale una frazione dei punti e ogni risposta sbagliata indicata ne toglie altrettanti; il punteggio di una domanda non scende mai sotto zero.

SOLUZIONI